



Security Assessment Draft Report



Compound Optimistic Governance

December 2025

Prepared for Compound Protocol

Table of content

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	4
Findings Summary	5
Severity Matrix	5
High Severity Issues	6
H-01 COMP Token Is Incompatible with GovernorVotes-Based Veto Governor	6
H-02 Queued Proposals Can Become Unexecutable	8
Medium Severity Issues	11
M-01 Unsafe Time Arithmetic in Veto Override Can Revert State	11
M-02 Council Governor state() Ignores Veto Governor State While Council Voting Is Still Active	14
M-03 Incompatibility with Proposer-Restricted Descriptions in Dual-Governor Forwarding	18
Low Severity Issues	21
L-01 Veto Governor Ignores GovernorTimelockControl.state()	21
L-02 Council Governor ETA Is Misleading	23
L-03 Low-Precision Council Token Supply Can Cause Quorum Rounding and Weaken Thresholds	26
L-04 ETH Transfers During Governance Execution Require a Pre-Funded Timelock	28
L-05 Incorrect Veto Period Extension Can Produce a Shorter-Than-Intended Extension	30
Informational Severity Issues	32
I-01. Ownership Management can be improved	32
I-02. MAX_TOKENS_PER_MEMBER Is Immutable and Limits Future Council Flexibility	33
I-03. Veto Extension Parameters Are Read Live Instead of Being Snapshot at Proposal Creation	34
I-04. Incorrect Documentation: clock() Returns Timestamp, Not Block NumberDescription	34
I-05. Misleading Parameter Name in GovernorExtendVetoPeriod.vetoThreshold()	35
I-06. Additional Notes & Suggestions	36
Disclaimer	37
About Certora	38

Project Summary

Project Scope

Project Name	Repository	Latest Commit Hash	Platform
Compound Optimistic Governance Protocol	Github Repo	Initial Commit - 6680eb4 Final Commit -	EVM

Project Overview

This document describes the manual code review of **Compound Optimistic Governance Protocol**.

The work was a **2** week effort undertaken from **December 16th, 2025** to **December 30th, 2025**.

The following contract list is included in our scope:

- `src/BasicCouncilGovernor.sol`
- `src/BasicCouncilVetoGovernor.sol`
- `src/CouncilERC20.sol`
- `src/extensions/GovernorAdmin.sol`
- `src/extensions/GovernorCouncilQueuing.sol`
- `src/extensions/GovernorVetoCountingSimple.sol`
- `src/extensions/GovernorExtendVetoPeriod.sol`
- `src/extensions/GovernorVetoGuardian.sol`
- `src/extensions/GovernorVetoOverride.sol`
- `src/extensions/GovernorVotesVetoThresholdFraction.sol`

The team performed a manual audit of all the Solidity smart contracts. During the manual audit, the Certora team discovered bugs in the Solidity smart contracts code, as listed on the following page.

Protocol Overview

The Compound Optimistic Governance system introduces a dual-governor design to enable faster protocol decision-making while preserving DAO oversight. A Council Governor allows a small, designated council to propose and approve actions quickly, while a Veto Governor gives the broader DAO the ability to reactively veto proposals under an optimistic, default-pass model. All successful proposals ultimately execute through a timelock.

The audit scope covers the on-chain governance pipeline, including the Council Governor, Veto Governor, CouncilERC20, timelock integration, and interactions with OpenZeppelin Governor internals. The review focuses on proposal lifecycle correctness, snapshot semantics, veto and extension logic, and state consistency across governors.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	-	-
High	2	-	-
Medium	3	-	-
Low	5	-	-
Informational	6	-	-
Total	16	-	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

High Severity Issues

H-01 COMP Token Is Incompatible with GovernorVotes-Based Veto Governor

Severity: High	Impact: High	Likelihood: Medium
Files: GovernorVetoOverride.sol	Status: Fixed	

Description: The veto governor relies on `GovernorVotes` / `IERC5805`, which requires the voting token to implement `getPastVotes()` and `getPastTotalSupply()`.

However, the intended DAO token, `COMP` ([0xc00e94cb662c3520282e6f5717214004a7f26888](#)), does not implement these functions. Instead, `COMP` exposes the legacy `getPriorVotes()` interface and does not support `getPastTotalSupply()`.

As a result, any code path that relies on `ERC5805` snapshot functions will revert when used with `COMP`.

JavaScript

```
function _getVotes(
    address account,
    uint256 timepoint,
    bytes memory /*params*/
) internal view virtual override returns (uint256) {
    return token().getPastVotes(account, timepoint);
}
```

Impact:

When configured with `COMP`,

- Proposal creation, quorum checks, veto threshold checks, and vote tallying will revert.

- The veto governor becomes non-functional.
- Governance actions are effectively DoSed at runtime.

Recommendations:

Use a COMP-compatible voting module (for example `GovernorVotesComp`) and implement veto threshold logic using COMP's `getPriorVotes()` semantics or fixed thresholds. Alternatively, clearly document that the veto governor requires an ERC5805-compliant token and cannot be used with COMP without modification

Customer's response: Fixed [here](#).

Fix Review: Fix looks good.

H-02 Queued Proposals Can Become Unexecutable

Severity: **High**

Impact: **High**

Likelihood: **Medium**

Files:
[GovernorVetoOverride.sol](#)

Status: Fixed

Description:

In the current implementation, the veto override mechanism determines its effectiveness dynamically inside `GovernorVetoOverride.state()` based on the current timestamp. This causes a proposal that was **validly overridden and queued** to later become **unexecutable**, even though no further governance action occurred.

JavaScript

```
function state(uint256 _proposalId) public view virtual override returns (ProposalState) {
    ProposalState _state = super.state(_proposalId);
    if (
        isVetoOverridden[_proposalId] && _state == ProposalState.Defeated
        && clock() - proposalDeadline(_proposalId) < vetoOverrideDuration
    ) return ProposalState.Succeeded;
    return _state;
}
```

The root cause is that override validity is **re-evaluated on `state()` call** using a time-based condition, rather than being locked in once the proposal successfully progresses to the queued state. As a result, a proposal can be correctly queued during the veto override window, but later blocked at execution time when the override window has elapsed.

Detailed Flow:

1. A proposal is created and voted on at the `BasicCouncilGovernor` level.
2. The proposal is forwarded to the `BasicCouncilVetoGovernor`.

3. The proposal is defeated in `BasicCouncilVetoGovernor` because the `vetoThreshold` is reached
4. The veto override role calls `overrideVeto(proposalId)`, marking the proposal as overridden.
5. Within the veto override window someone calls `queue()`.

JavaScript

```
proposalDeadline < clock() < proposalDeadline + vetoOverrideDuration
```

6. During `queue()`:
 - `state()` is evaluated.
 - The override condition is satisfied.
 - `state()` returns `Succeeded`.
 - The proposal is successfully queued in the timelock.
 - At this point, the proposal should be executed after the timelock delay is passed.
7. Time advances beyond `proposalDeadline + vetoOverrideDuration`.
8. Someone attempts to call `execute()` on the already-queued proposal.
9. During `execute()`:
 - `state()` is evaluated again.
 - `clock() - proposalDeadline(_proposalId) < vetoOverrideDuration` will not be true
 - `state()` returns `Defeated`.
10. Inside the `GovernorCouncilQueuing.state()`, it will be evaluated as `Canceled`. Execution is blocked permanently, even though the proposal was previously queued correctly.

Recommendations:

`GovernorVetoOverride.state()` should explicitly detect when a proposal has already been queued by checking:

JavaScript

```
proposalEta(proposalId) != 0
```

and **return** `ProposalState.Queued` **regardless of whether earlier state logic would classify it as** `Defeated`. This ensures that once a proposal has successfully passed all checks and entered the timelock queue, its queued state is **final and cannot be superseded**.

Customer's response: Fixed [here](#)

Fix Review: Fix looks good.

Medium Severity Issues

M-01 Unsafe Time Arithmetic in Veto Override Can Revert State

Severity: **Medium**

Impact: **Medium**

Likelihood: **Medium**

Files:
[GovernorVetoOverride.sol](#)

Status: Fixed

Description:

The veto override mechanism contains multiple issues in its time-based state logic that cause incorrect behavior and deviate from the intended governance semantics.

First, the `state()` function performs unchecked time arithmetic using `clock() - proposalDeadline`. If an override is applied before the proposal deadline and the proposal state is defeated, this subtraction underflows and causes `state()` to revert. As a result, the proposal's state becomes unreadable during this period.

Second, the override is only effective in a narrow, implicit window defined by

`proposalDeadline < clock() < proposalDeadline + vetoOverrideDuration`.

This window is not enforced when `overrideVeto()` is called. As a result, overrides can be applied at times when they will never take effect, leading to confusing and fragile behavior that is difficult to reason about in practice.

JavaScript

```
if (
  isVetoOverridden[_proposalId] &&
  _state == ProposalState.Defeated &&
  clock() - proposalDeadline(_proposalId) < vetoOverrideDuration
) {
  return ProposalState.Succeeded;
}
```

```
}
```

Case 1: Override Exercised Before Proposal Deadline

Example parameters:

None

```
proposalDeadline = 100  
vetoOverrideDuration = 10
```

Scenario:

- Proposal is Active.
- Guardian vetoes early, forcing `_state == Defeated`.
- Override role calls `overrideVeto()` at `clock = 95`.

Evaluation:

None

```
95 - 100 → underflow → revert
```

Result:

- `state()` reverts instead of returning a valid state.
- `queue()` cannot be called.
- Proposal is temporarily unusable until `clock() >= proposalDeadline`.

Case 2: Override Exercised Within Override Window (Only Working Case)

Scenario:

- Override applied at `clock = 105`.

Evaluation:



JavaScript

```
105 - 100 = 5 < 10
```

Result:

- `state()` returns `Succeeded`.
- Proposal can be queued.

This is the **only** window in which the override has any effect.

Impact

- `state()` can revert due to underflow, breaking governance automation and UI integrations.
- Overrides applied before the proposal deadline are unusable and can temporarily brick proposal state queries.

Recommendations:

- Enforce the override window at the time `overrideVeto()` is called, reverting if it is outside the allowed window.
- Clearly document the override timing rules so users do not rely on implicit or non-obvious behavior.

Customer's response: Fixed [here](#)

Fix Review: Fix looks good.

M-02 Council Governor state() Ignores Veto Governor State While Council Voting Is Still Active

Severity: **Medium**

Impact: **Medium**

Likelihood: **Medium**

Files:
[GovernorCouncilQueueing.sol](#)

Status: Not Fixed

Description:

The Council Governor's `state()` resolution follows a layered inheritance order:

JavaScript

```
BasicCouncilGovernor.state
  → GovernorVotesSuperQuorumFraction.state
    → GovernorSuperQuorum.state
      → GovernorCouncilQueueing.state
        → Governor.state
```

Within `GovernorCouncilQueueing.state`, the veto governor's state is consulted only if the state returned by `Governor.state()` is `Queued`. If `Governor.state()` returns any other value (e.g., `Active`), that state is propagated upward unchanged, and the veto governor is not queried at all.

JavaScript

```
function state(uint256 _proposalId) public view virtual override returns (ProposalState) {
    ProposalState _currentState = super.state(_proposalId);

    if (_currentState != ProposalState.Queued) return _currentState;
    if (_checkVetoGovernorStateBitmap(
        _proposalId,
        _encodeStateBitmap(ProposalState.Pending) | _encodeStateBitmap(ProposalState.Active)
        | _encodeStateBitmap(ProposalState.Queued) |
        _encodeStateBitmap(ProposalState.Succeeded)
    )) {
        return ProposalState.Queued;
    } else if (_checkVetoGovernorStateBitmap(
```

```
        _proposalId, _encodeStateBitmap(ProposalState.Executed)
    )) {
        // Fallback for proposals executed directly on the veto governor or the timelock
        return ProposalState.Executed;
    } else {
        return ProposalState.Canceled;
    }
}
```

This creates an inconsistency when **super-quorum logic allows early queuing while the council voting period is still active**. Consider a scenario:

1. Council voting starts.
2. Super-quorum is reached early and `_voteSucceeded == true`.
3. The proposal is queued immediately in the Council Governor and forwarded to the Veto Governor.
4. The Veto Governor proposal becomes active and may even be **defeated early** (e.g., via guardian veto or reaching veto threshold).
5. Meanwhile, because the council voting period has not ended, `Governor.state()` continues to return `Active`.
6. Since the state is not `Queued`, `GovernorCouncilQueuing.state` **does not consult the Veto Governor** and propagates `Active` upward.
7. As a result, `BasicCouncilGovernor.state()` reports an incorrect state of `Queued` even though the veto governor has the proposal as defeated. Ideally, it should have been `Cancelled`.

In effect, **the Council Governor ignores the veto governor's state until the council voting duration ends**, even though the proposal has already been proposed, evaluated, and defeated in the veto layer. This behavior is especially likely in small councils (e.g., 5–7 members), where super-quorum can be reached very quickly, long before the voting period expires.

POC

Place below test in `BasicCouncilGovernor.integration.t.sol`:

JavaScript

```
function test_M02_CouncilStateStaysQueuedAfterVetoDefeatDuringCouncilVoting() public {
    // Propose
    vm.prank(councilMembers[0]);
    uint256 _proposalId = councilGovernor.propose(targets, values, calldatas, description);
    // Warp past voting delay
    vm.warp(block.timestamp + councilGovernor.votingDelay() + 1);
    // Meet superQuorum (7) to succeed early
    for (
        uint256 _i = 0;
        _i < councilGovernor.superQuorum(councilGovernor.proposalSnapshot(_proposalId));
        _i++
    ) {
        vm.prank(councilMembers[_i]);
        councilGovernor.castVote(_proposalId, 1); // 1 = For
    }
    assertEq(uint8(councilGovernor.state(_proposalId)),
uint8(IGovernor.ProposalState.Succeeded));
    // Queue/forward while the council voting period is still active
    councilGovernor.queue(targets, values, calldatas, descriptionHash);
    assertEq(uint8(councilGovernor.state(_proposalId)), uint8(IGovernor.ProposalState.Queued));
    // Veto guardian defeats the veto-layer proposal immediately (veto proposal is Pending here)
    vm.prank(vetoGuardian);
    vetoGovernor.vetoByGuardian(_proposalId);
    assertEq(uint8(vetoGovernor.state(_proposalId)), uint8(IGovernor.ProposalState.Defeated));
    // BUG: council governor does not reflect the veto defeat yet.
    assertEq(uint8(councilGovernor.state(_proposalId)), uint8(IGovernor.ProposalState.Queued));
}
```

Impact

- The Council Governor can report proposals as **Queued** even after they are defeated in the Veto Governor which can mislead the council members and other users.
- Depending upon the **voteDuration** in VetoGovernor, execution attempts routed through the Council Governor may revert unexpectedly.

Recommendation

Update the Council Governor's state resolution logic to **always consult the Veto Governor once the proposal has been forwarded**, regardless of whether the council voting period has ended.

Customer's response:

Fix Review

M-03 Incompatibility with Proposer-Restricted Descriptions in Dual-Governor Forwarding

Severity: **Medium**

Impact: **High**

Likelihood: **Low**

Files:
[BasicCouncilVetoGovernor.sol](#)

Status: Fixed

Description: OpenZeppelin Governor supports an opt-in proposer restriction mechanism via proposal descriptions ending with:

```
JavaScript
#proposer=0xABC...
```

When present, `Governor.propose` enforces that:

- Only the specified address may submit the proposal.
- Any submission by a different address reverts, even if calldata and description are identical.
- This mechanism is designed to prevent:
 - **Mempool frontrunning**, where an attacker submits an identical proposal first and becomes the proposer.
 - **Proposal griefing or perpetual cancellation**, where proposer privileges are abused to block or repeatedly cancel proposals before meaningful voting occurs. The check is enforced in `Governor.propose` via:

```
JavaScript
_isValidDescriptionForProposer(msg.sender, description)
```

However, it wouldn't work in Dual-Governor Architecture.

In the Compound Optimistic Governance flow:

1. A council member EOA submits a proposal to the BasicCouncilGovernor.
2. After passing council voting, the proposal is forwarded to `BasicCouncilVetoGovernor` via `GovernorCouncilQueuing`.
3. During forwarding:
 - `BasicCouncilVetoGovernor.propose` is invoked by the `CouncilGovernor` contract, not by the original council member.
 - The proposal description string is forwarded unchanged.

If the original proposal description contains:

JavaScript

```
#proposer=OxCouncilMember
```

then veto governor proposal creation fails because:

- `msg.sender == address(CouncilGovernor)`
- `recoveredProposer == OxCouncilMember`
- `_isValidDescriptionForProposer` returns false
- `propose` reverts

This revert happens **after the council proposal has already succeeded**.

Impact:

This creates a hard governance deadlock at the council → veto boundary:

- A proposal can successfully pass council voting but can never be queued into the veto governor.
- The proposal can never reach the timelock or execution.
- There is no on-chain recovery path once this state is reached.

Critically, this means:

- The proposer-restriction mechanism cannot be used at all in the current implementation.
- Any attempt to use OpenZeppelin's documented frontrunning protection will brick the proposal pipeline.

At the same time, if proposer-restricted descriptions are not used, proposals are exposed to the very front-running and cancellation risks that the mechanism is meant to mitigate. In particular:

- Any address that meets the proposal threshold (including a council member holding a single council token, if thresholds permit) can front-run proposal submission.
- A malicious or opportunistic proposer can submit and then cancel or otherwise interfere with time-sensitive proposals before meaningful voting occurs.

This is especially dangerous for time-critical or emergency proposals, such as:

- Risk or parameter updates

- Oracle or market configuration changes
- Incident response or mitigation actions

In such cases, a malicious council member can exploit this incompatibility to halt or indefinitely delay important governance actions.

Recommendations:

Override proposer validation in the veto governor

- Override `_isValidDescriptionForProposer` to always return true.
- Document this as a deliberate deviation from OpenZeppelin Governor semantics.

This is safe because the veto governor does not accept proposals from arbitrary EOAs. It only accepts proposals forwarded from the council governor. Forwarded proposals are already uniquely identified by (targets, values, calldatas, descriptionHash) and must correspond to an existing council proposal in the correct Succeeded state before they can be submitted.

As a result, the original security guarantees of proposer-restricted descriptions, namely preventing front-running and proposal griefing, are already enforced structurally by the council → veto gating, making proposer validation at the veto layer unnecessary.

Customer's response: Fixed [here](#).

Fix Review: Fix looks good.

Low Severity Issues

L-01 Veto Governor Ignores <code>GovernorTimelockControl.state()</code>		
Severity: Low	Impact: Low	Likelihood: Low
Files:	Status: Not Fixed	

BasicCouncilVetoGovernor.sol		
--	--	--

Description

In the current implementation, `BasicCouncilVetoGovernor` overrides `state()` and derives proposal status exclusively from veto-related extensions, without composing with `GovernorTimelockControl.state()`. As a result, the veto governor never consults the timelock's internal operation state when determining proposal finality.

This creates a mismatch when a proposal is cancelled directly on the timelock by an address holding `CANCELLER_ROLE`. In that scenario, the timelock correctly marks the scheduled operation as cancelled. However, because `GovernorTimelockControl.state()` is never invoked, this cancellation is not reflected in `BasicCouncilVetoGovernor.state()`.

Depending on the timing and interaction with other extensions, the veto governor may continue to report the proposal as `Queued`, or in some specific cases as `Defeated` after it was previously queued (see Issue [H-02](#)). In the latter case, the proposal may even become eligible to be queued again if the Queue is called inside `vetoOverrideDuration`, despite the underlying time-lock operation having already been cancelled.

The council governor mirrors this incorrect state and propagates it unchanged.

Impact

- `state()` can continue to report `Queued` or `Defeated` even after the timelock operation has been cancelled.
- In specific edge cases, a cancelled proposal may become re-queueable.
- Off-chain automation, UIs, and governance operators may incorrectly assume a proposal is executable or in a valid lifecycle stage.
- This violates an expected governance invariant:
 - Once a timelock operation is cancelled, the proposal should be reflected as terminal and non-executable at the governor level.

Recommendation

Consider invoking `GovernorTimelockControl.state()` in `BasicCouncilVetoGovernor.state()`'s hierarchy.

Customer's response: Fixed [here](#) and [here](#)

Fix Review: Fix looks good

L-02 Council Governor ETA Is Misleading

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files:
[GovernorCouncilQueuing.sol](#)

Status: Fixed

Description:

The Council Governor records an ETA for a proposal at queue time using the Veto Governor's `proposalDeadline`. However, `proposalDeadline` represents the end of the veto voting window, not the time at which the proposal becomes executable.

JavaScript

```
function _queueOperations(
    uint256 _proposalId,
    address[] memory _targets,
    uint256[] memory _values,
    bytes[] memory _calldatas,
    bytes32 /* _descriptionHash */
) internal virtual override returns (uint48) {

    councilVetoGovernor.propose(_targets, _values, _calldatas,
proposalDescriptions[_proposalId]);

    delete proposalDescriptions[_proposalId];

    return uint48(councilVetoGovernor.proposalDeadline(_proposalId));
}
```

JavaScript

```
function queue(
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    bytes32 descriptionHash
) public virtual returns (uint256) {

    /// Code

    uint48 etaSeconds = _queueOperations(proposalId, targets, values, calldatas,
descriptionHash);

    if (etaSeconds != 0) {
        _proposals[proposalId].etaSeconds = etaSeconds;
    }
}
```

```
        emit ProposalQueued(proposalId, etaSeconds);
    } else {
        revert GovernorQueueNotImplemented();
    }

    return proposalId;
}
```

Execution readiness is actually governed by the **Timelock**, which independently computes its own ETA as `block.timestamp + minDelay` when the proposal is queued in the timelock. Separately, the Veto Governor may later extend its voting deadline via the veto extension mechanism, but these extensions are not propagated back to the Council Governor. As a result, there are three independent and unsynchronized timelines:

- **Council Governor ETA** – derived once from `proposalDeadline`
- **Veto Governor deadline** – may be extended after queuing
- **Timelock ETA** – the only timeline that determines executability

Using `proposalDeadline` as the Council Governor’s ETA is therefore semantically incorrect as [ETA](#) is intended to signal execution readiness. Separately, even if the use of `proposalDeadline` is intentional at the council layer, it does not reflect veto deadline extensions and quickly becomes stale.

Impact

- The Council Governor may report proposals as executable earlier than they actually are.
- Calls to `execute()` routed through the Council Governor can revert due to:
 - active or extended veto windows, or
 - unmet timelock delays.
- Off-chain tooling, automation, and UIs relying on the Council Governor’s ETA may behave incorrectly.
- Proposal lifecycle observability becomes confusing, with ETA no longer meaning “ready for execution.”

Recommendation

Clarify the semantics of ETA at the Council Governor layer and avoid treating it as an execution-readiness signal. Alternatively, consider overriding `CouncilGovernor.proposalEta` to directly surface the Veto Governor's ETA:

None

```
function proposalEta(uint256 proposalId) public view virtual returns
(uint256) {
    return councilVetoGovernor.proposalEta(proposalId);
}
```

Under this approach, the function would return `0` until the proposal is queued in the timelock via the Veto Governor, and then return the correct timelock execution ETA once available. This aligns the Council Governor's ETA with actual executability while avoiding reliance on veto voting deadlines. At minimum, this behavior should be clearly documented so integrators and tooling do not infer executability from the Council Governor's ETA.

Customer's response: Fixed [here](#).

Fix Review: Fix looks good.

L-03 Low-Precision Council Token Supply Can Cause Quorum Rounding and Weaken Thresholds

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files:
[CouncilERC20.sol](#)

Status: Fixed

Description:

In the current scripts and tests, each council member is minted only 1 wei of CouncilERC20, and `MAX_TOKENS_PER_MEMBER` is also set to 1. This results in an extremely small total token supply, equal to the number of council members. Quorum is computed using OpenZeppelin's fractional quorum logic:

JavaScript

```
quorum = floor( token().getPastTotalSupply(timepoint) * quorumNumerator(timepoint) /  
quorumDenominator() );
```

Because the number of council members is expected to be small, this low-precision supply amplifies integer rounding effects. As a result, quorum and super-quorum thresholds can effectively resolve fewer council members than intended.

Consider the following configuration:

- Baseline example: 3 out of 6 council members
 - Council members: 6
 - Total supply: 6
 - Intended quorum: **3 out of 6**
 - Configuration: `quorumNumerator = 50, quorumDenominator = 100`

Calculation:

JavaScript

```
quorum = floor( 6 * 50 / 100 ) = 3
```

This matches the intended threshold.

Same configuration, but a member is added. With the same quorum parameters and total supply increased to 7:

JavaScript

```
quorum = floor( 7 * 50 / 100 ) = floor( 3.5 ) = 3
```

Although “50% quorum” would intuitively suggest **4 out of 7**, the effective quorum remains **3** (42%) due to rounding down.

This weakens the approval threshold without any explicit governance change.

The same issue applies to super-quorum thresholds, where rounding can allow early finalization with fewer votes than intended.

Recommendation

This behavior should be clearly documented so quorum parameters are chosen with rounding effects in mind.

As an optional improvement, consider minting council tokens with higher precision (for example, $1e18$ per member) by introducing `MIN_TOKENS_PER_MEMBER` to reduce rounding errors and make fractional quorum calculations more closely reflect intended council-member thresholds.

Customer’s response: Fixed [here](#).

Fix Review: Fix looks good.

L-04 ETH Transfers During Governance Execution Require a Pre-Funded Timelock

Severity: Low	Impact: Low	Likelihood: Low
Files: GovernorCouncilQueueing.sol	Status: Fixed	

Description: There are two common patterns for supporting ETH transfers during governance execution:

1. Forwarding ETH via `msg.value` through `execute()`, and
2. Executing transfers from a pre-funded timelock balance.

The current optimistic governance implementation exclusively follows the second model. In `BasicCouncilGovernor`, `GovernorCouncilQueueing._executeOperations` calls `councilVetoGovernor.execute(...)` without forwarding `msg.value`. Consequently, when the veto governor executes via `GovernorTimelockControl`, `msg.value` is always zero. Any ETH specified in `values[ ]` must therefore be paid entirely from the timelock's existing balance.

JavaScript

```
function _executeOperations(
  uint256, /*_proposalId*/
  address[] memory _targets,
  uint256[] memory _values,
  bytes[] memory _calldatas,
  bytes32 _descriptionHash
) internal virtual override {
  councilVetoGovernor.execute(_targets, _values, _calldatas, _descriptionHash);
}
```

This behavior is implicit and not enforced or documented at the interface level.

Impact:

- ETH transfers will fail at execution time if the timelock is not sufficiently pre-funded.
- Callers cannot attach ETH to `execute()` to fund transfers, which may surprise integrators and operators.
- Off-chain automation or governance tooling that assumes `msg.value` forwarding may misconfigure executions.

Recommendations:

Explicitly document that governance execution relies on a pre-funded timelock for ETH transfers. If broader compatibility is desired, consider forwarding `msg.value` through the council and veto governors to the timelock, otherwise, clearly specify the pre-funding requirement to avoid execution failures.

Customer's response: Fixed [here](#).

Fix Review: Fix looks good.

L-05 Incorrect Veto Period Extension Can Produce a Shorter-Than-Intended Extension

Severity: Low	Impact: Low	Likelihood: Low
Files: GovernorExtendVetoPeriod.sol	Status: Fixed	

Description:

The veto-period extension mechanism appears to set the **extended deadline** as:

- `extendedDeadline = clock() + votingPeriodExtension()`

This gives a *minimum remaining time from the moment the threshold is reached*, not a *fixed extension added to the existing deadline*. As a result, depending on when the threshold is triggered, the effective extension can be smaller than expected for voters and integrators.

Example

Assume:

- Proposal submitted = 500
- Vote start = 500 + voting delay = 500 + 100 = 600
- Original `proposalDeadline` = 600 + `initialVotingPeriod` = 600 + 400 = 1,000

Case A: A low `votingPeriodExtension`

- `votingPeriodExtension` = 200
- Threshold reached at `clock()` = 700
- Computed: `extendedDeadline` = 700 + 200 = 900
- Since 900 < 1,000, extension is ignored and deadline stays 1,000 even though extension was triggered

Case B: A high `votingPeriodExtension`

- `votingPeriodExtension` = 600
- Threshold reached at `clock()` = 900
- Computed: `extendedDeadline` = 900 + 600 = 1500
- New deadline becomes 1500
- Effective extension beyond original deadline: 1500 - 1,000 = 500 (not 600)

So even though the “extension parameter” is 600, the proposal only gets 500 extra time past the original deadline

Recommendation

Extend from the existing deadline, not from `clock()`, e.g.:

- `extendedDeadline = proposalDeadline(proposalId) + votingPeriodExtension()`

Customer’s response: Fixed [here](#).

Fix Review: Fix looks good.

Informational Severity Issues

I-01. Ownership Management can be improved

Description:

`BasicCouncilVetoGovernor`, and `CouncilERC20` all use OpenZeppelin's single-step `Ownable` module. In the current setup, the owner is intended to be the main DAO governor. However, `Ownable` allows both immediate ownership transfers and calling `renounceOwnership()`, which permanently sets the owner to `address(0)`.

Because ownership controls governance-critical configuration and recovery paths, accidentally or mistakenly invoking `renounceOwnership()` would permanently disable administrative functionality. Similarly, single-step ownership transfers increase the risk of misconfiguration or irreversible mistakes during handover.

Recommendation:

Improve ownership safety by:

- Migrating to `Ownable2Step` to require explicit acceptance of ownership transfers.
- Overriding and disabling `renounceOwnership()` to prevent accidental or irreversible loss of admin control.

These changes reduce operational risk while preserving controlled and deliberate ownership transitions.

Customer's response:

Fix Review:

I-02. MAX_TOKENS_PER_MEMBER Is Immutable and Limits Future Council Flexibility

Description:

In `CouncilERC20`, the `MAX_TOKENS_PER_MEMBER` parameter is declared as `immutable`, which

fixes the maximum number of tokens that can be minted to a single council member at deployment time.

This value directly influences council voting power distribution and effectively encodes assumptions about how council membership and weighting should work long term. Because it is immutable, this constraint cannot be adjusted even if governance requirements or council structure evolve.

Recommendation:

Consider making `MAX_TOKENS_PER_MEMBER` configurable via governance rather than immutable.

Customer's response:

Fix Review:

I-03. Veto Extension Parameters Are Read Live Instead of Being Snapshot at Proposal Creation

Description:

The veto voting period extension logic reads `votingPeriodExtension()` and `minorVetoExtensionThresholdPct()` directly from storage at the time `_tallyUpdated` is executed. These parameters are not snapshotted at proposal creation and can be modified while a proposal is already active. This differs from other governance parameters, such as quorum and super-quorum settings, which are correctly snapshotted at the proposal snapshot. As a result, different parts of the voting logic follow different temporal semantics.

Recommendation:

Consider snapshotting `votingPeriodExtension` and `minorVetoExtensionThresholdPct` at proposal creation, similar to quorum-related parameters, and use the snapshotted values throughout the proposal lifecycle. This aligns veto extension behavior with existing snapshot-based governance semantics and improves predictability.

Customer's response:

Fix Review:

I-04. Incorrect Documentation: `clock()` Returns Timestamp, Not Block NumberDescription

Description:

The NatSpec comment states that [CouncilERC20.clock\(\)](#) returns the current block number, but the implementation actually returns `block.timestamp` cast to `uint48`. This is a documentation mismatch. In OpenZeppelin Governor semantics, the meaning of `clock()` is critical, as it defines the timepoint used for snapshots, voting periods, and deadlines. Mislabeling it as a block number can mislead integrators into assuming block-based governance when the system is in fact timestamp-based.

JavaScript

```
/// @notice Returns the current block number as the clock.  
function clock() public view override returns (uint48) {  
    return uint48(block.timestamp);  
}
```

Recommendation:

Update the NatSpec comment to explicitly state that `clock()` returns the current block timestamp.

Customer's response: Fixed [here](#)

Fix Review: Fix looks good.

I-05. Misleading Parameter Name in GovernorExtendVetoPeriod.vetoThreshold()

Description:

In [GovernorExtendVetoPeriod.sol](#), the abstract function is declared as:

None

```
function vetoThreshold(uint256 _proposalId) public view virtual returns
(uint256 _vetoThreshold);
```

While the ABI signature is still `vetoThreshold(uint256)`, naming the parameter `_proposalId` is misleading because the rest of the system's veto-threshold math is naturally defined over the proposal snapshot timepoint (`proposalSnapshot(proposalId)`), not over a proposal id itself. This can cause implementers/integrators to mistakenly implement `vetoThreshold` based on a proposal id, resulting in incorrect threshold evaluation and potentially wrong veto-period extension behavior.

Recommendation

Rename the parameter to reflect intended semantics, e.g. `vetoThreshold(uint256 timepoint)`

Customer's response: Fixed [here](#).

Fix Review: Fix looks good.

I-06. Additional Notes & Suggestions

Description:

1. While queueing batches via OpenZeppelin's `GovernorTimelockControl`, the default `predecessor = 0` is used. This schedules proposals without dependency on prior proposals, allowing out-of-order execution.

JavaScript

```
_timelockIds[proposalId] = _timelock.hashOperationBatch(targets, values, calldatas, 0, salt);
```

- **Suggestion:** If strict execution order is intended, update the predecessor logic, otherwise, document the behavior clearly so users understand that proposal

execution order is not enforced.

2. According to the [Governor](#) contract documentation, [_executor\(\)](#) should return the address that actually executes proposal actions (typically the timelock).

JavaScript

```
/**
 * @dev Address through which the governor executes action. Will be overloaded by module
that executes actions
 * through another contract such as a timelock.
 */
```

- However, [BasicCouncilGovernor](#) returns [BasicCouncilVetoGovernor](#) address instead of the timelock, which may mislead integrators into assuming the veto governor directly executes proposals.
- **Suggestion:** Return the timelock address in `_executor()` or document the deviation clearly to avoid confusion about which contract ultimately executes proposal actions.

Customer's response:

Fix Review:

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.